ARGUS · SENIOR DESIGN 2026

ARGUS Federated Learning in Production

The schedule, model versions, rollback, deployment and monitoring

Federated learning in production

How ARGUS runs its learning loop unattended: the schedule that starts each round, the version every model gets, how a bad version is rolled back, and how to deploy and watch it. Current as of 2026-10-05 on `feat/threat-detection`.

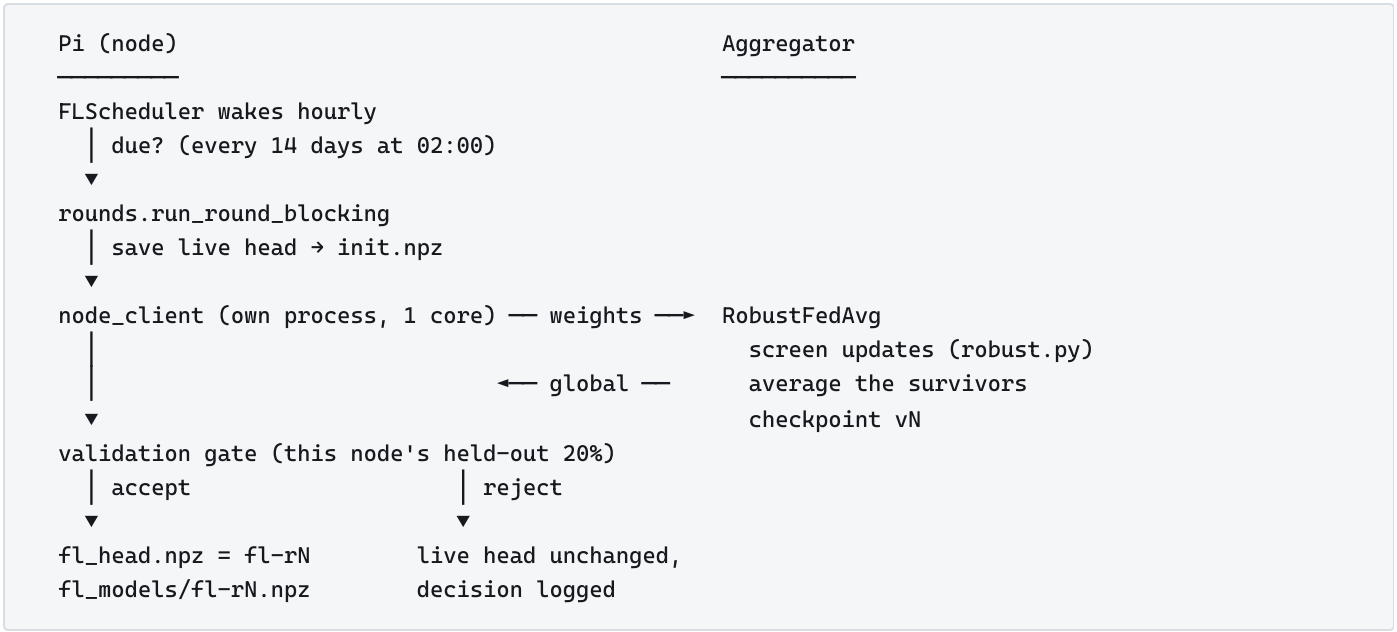
Part of this describes code changed on 2026-10-05 and not yet committed (robust aggregation in the servers, the version archive and rollback, the cron retry fix, the aggregator version counter, the systemd unit). Section 9 says exactly what has been verified and what has not.

---

1. What runs where

Machine	Process	Started by	Job
Each Pi (a node)	`main.py` (FastAPI, port 8000)	`deploy/argus-node.service` (systemd)	Detection, event capture, labelling, **the schedule**, local training, the validation gate, the model archive
The aggregator (laptop or server)	`central_server/fl_aggregator.py` (Flower on 8080, REST on 8090)	`scripts/run_central_server.ps1`, or `docker compose up -d` in `central_server/`	Waits for nodes, screens and averages their updates, versions and checkpoints the global model

There is no separate cron daemon. The schedule is a task inside the node process (`src/federated/scheduler.py`), which is why the node has to run as a service: if the node is not running, no round happens.



Only weights cross the network. No video, stills or feature vectors.

---

2. The schedule

`FLScheduler` runs only when `FL_ENABLED=true`.

Rule	How
When	Every `FL_ROUND_INTERVAL_DAYS` (default 14) at `FL_ROUND_HOUR` (default 2, i.e. 02:00 local)
Survives reboots	The last completed round is written to `data/fl_state.json`. A node rebooted weekly still trains every 14 days instead of restarting the countdown
Catches up	A round that came due while the node was off runs 5 minutes after it starts
Wakes hourly	It never sleeps 14 days at once, so suspend, NTP clock jumps on an RTC-less Pi, and shutdown are all handled
Retries failures	A round that did not happen (aggregator unreachable, timed out, fewer than 200 labelled samples) does not advance the schedule, so it is retried at the next hourly wake
Counts rejections	A round where the gate rejected the model **did** happen. It advances the schedule; retrying hourly would not change the answer
Never blocks the camera	Training runs as a separate process pinned to one BLAS thread (`ARGUS_TRAIN_THREADS`)

Fixed 2026-10-05. The retry rule above was not true before. The scheduled path calls `run_round_blocking`, which reports an unreachable aggregator as a result (`{"accepted": false, ...}`) rather than raising. The scheduler only treated exceptions as failures, so an unreachable aggregator was recorded as a completed round

and the retry was pushed 14 days out. `run_round_blocking` now returns `completed: true|false`, and the scheduler raises on `completed: false`. Two tests in `tests/test_fl_schedule.py` pin both cases through the real runner path.

Not yet true: the "only while idle" check. `is_idle()` defers a round if motion was seen in the last 10 minutes, but nothing calls `notify_motion()`, so it always reads idle. At 02:00 this rarely matters, and the round runs in its own process either way.

Change the schedule in `deploy/node.env` (or the environment) and restart the node:

```
FL_ROUND_INTERVAL_DAYS=7
FL_ROUND_HOUR=3
```

---

3. A scheduled round, end to end

`src/federated/rounds.py`, `run_round_blocking(app)`. The manual trigger (`POST /api/federated/round`) follows the same steps, so the 02:00 path is the one the demo exercises.

1. Require 200 labelled samples; otherwise return `completed: false`.
2. Write the live head to `data/fl_rounds/<timestamp>/init.npz`.
3. Launch `python -m src.federated.node_client` as its own process. It registers with the aggregator's REST API (`FL_CENTRAL_API`), then joins the Flower round at `FL_SERVER_URL`: trains on this node's labelled training split, sends weights plus its `node_id`, and saves the aggregated model the server returns as `global.npz`.
4. On the aggregator, `RobustFedAvg` screens every update against the global model it sent out (structural, norm, direction, trimmed mean; direction and trimming need three or more nodes) and averages the survivors. If nothing survives, the global model does not change and no checkpoint is written.
5. Back on the node, `global.npz` goes through the validation gate.
6. The result is logged and the schedule advances.

Everything from the round stays in `data/fl_rounds/<timestamp>/` (`init.npz`, `global.npz`, `events.jsonl`) so any round can be inspected afterwards.

---

4. Model versions

There are two version sequences, one per node and one on the aggregator. They count different things and do not share numbers.

4.1 On each node

Every model that becomes live gets the next number. The counter only goes forward, including on rollback.

Label	Meaning
`untrained`	No model accepted yet
`local-rN`	Accepted from local training (`POST /api/federated/head/train`)
`fl-rN`	Accepted from a federated round
`rollback-rN`	An archived version made live again by hand

File	Holds
`data/fl_head.npz`	The live head, restored at startup
`data/fl_models/<label>.npz`	Every accepted version, newest 20 kept (`MAX_ARCHIVED`). Under 1 MB each
`data/fl_model_state.json`	Current version and label, and the last 50 gate decisions, accepted or not, with both models' scores and the reason for any rejection

The gate. A candidate replaces the live head only if, on this node's own held-out 20% of labelled events:

- its balanced accuracy is not more than 0.01 below the live head's, and
- when balanced accuracy is level, its loss is not more than 5% worse.

The second rule was added 2026-10-05. In the 2026-09-20 hardware run a model with worse loss (0.1941 → 0.2529) was accepted at 0.75 against 0.75; it would now be rejected. A candidate that clearly improves balanced accuracy is taken regardless of loss. The first model is accepted outright.

Rollback (added 2026-10-05). The gate stops a bad model it can measure. Rollback covers the one it cannot: a version that passed the gate and then behaved badly on the camera.

```
# PI
curl -s localhost:8000/api/federated/head/versions | python -m json.tool
curl -sX POST localhost:8000/api/federated/head/rollback \
     -H 'Content-Type: application/json' -d '{"label": "fl-r3"}'
```

The restored weights become a new version (`rollback-r6` , say), so the history shows what was replaced, by what, and when. Rollback refuses with 409 while a manually started round is running and 404 for a label that is not archived.

4.2 On the aggregator

Every round that produces a model writes a checkpoint and increments the global version:

File	Holds
`central_server/checkpoints/model_round_<RRRR>_v<N>.npz`	The global model after that round
`central_server/checkpoints/model_latest.npz`	A copy of the newest, used as the starting point when the server restarts

Fixed 2026-10-05. The counter lived only in memory and started at 0 on every launch. A restarted server's first checkpoint was named exactly like the previous run's round-1 file and replaced it, and two different models answered to the same version. `resume_model_version()` now scans the checkpoint folder at startup and continues from the highest version on disk. Two tests in `tests/test_model_versions.py` cover it.

The round numbers in file names restart each session (the server serves sessions back to back so a node on a 14-day schedule still finds it); the version number is the one that identifies a model.

```
# LAPTOP
curl -s localhost:8090/api/model/checkpoints   # last 20, newest last
curl -s localhost:8090/api/training/history    # per round: clients, loss, accuracy, screening
report
curl -sO localhost:8090/api/model/latest       # download the current global model
```

Per-round history is held in memory, so it covers the current server process only. The checkpoints are the durable record.

---

5. Deploying it

5.1 The node, as a service

```
# PI, from ~/argus
cp deploy/node.env.example deploy/node.env
nano deploy/node.env          # NODE_NAME, FL_SERVER_URL, FL_CENTRAL_API
sudo cp deploy/argus-node.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now argus-node
journalctl -u argus-node -f
```

The unit assumes user `pi` and the repo at `/home/pi/argus`. It restarts the node 10 seconds after a crash and gives it 45 seconds to stop cleanly, because an unclean stop already corrupted one SD card. `deploy/node.env` is git-ignored since it can hold a password.

Every node needs its own `NODE_NAME`. The aggregator identifies updates by it; two nodes left on the default would be told apart only by a `#1` suffix in the screening report.

5.2 The aggregator

Windows laptop, foreground:

```
.\scripts\run_central_server.ps1 -MinClients 2
```

Allow the ports once, from an elevated PowerShell, or the Pi cannot connect:

```
New-NetFirewallRule -DisplayName 'ARGUS FL' -Direction Inbound -Protocol TCP -LocalPort 8080,8090 -
Action Allow
```

As a long-running service with Prometheus and Grafana, on any Docker host:

```
cd central_server
docker compose up -d --build     # restart: unless-stopped
```

The image now builds from the repository root so it can include `src/federated/robust.py` and `robust_strategy.py`; `central_server/Dockerfile.dockerignore` keeps the build context to the five files it copies.

5.3 Settings that matter in production

Setting	Where	Production value
`FL_ENABLED`	node	`true` (`run_node.sh` defaults it to `false` for demos)
`FL_SERVER_URL`, `FL_CENTRAL_API`	node	The aggregator's address
`NODE_NAME`	node	Unique per node
`ARGUS_MIN_CLIENTS` / `-MinClients`	aggregator	The number of nodes actually deployed. With 1 the server "federates" one node with itself
`ARGUS_AGGREGATION` / `-Aggregation`	aggregator	`robust`. `fedavg` is only for the comparison study
`ARGUS_AUTH_TOKEN` / `-Token`	aggregator	Set it whenever the aggregator is reachable beyond the LAN

---

6. Watching it

Question	Where
When is the next round on this node?	`GET /api/federated/status` (node)
What model is live, and why was the last candidate accepted or rejected?	`GET /api/federated/head` (node), `last_decision`
Which versions can I roll back to?	`GET /api/federated/head/versions` (node)
How did the last manual round go?	`GET /api/federated/round` (node)
Which nodes are registered and when did they last train?	`GET /api/nodes` (aggregator)
Who was rejected by screening, and why?	`GET /api/training/history` (aggregator), the `aggregation` field of each round
Trends	Prometheus on `:8090/metrics`; Grafana in the Docker stack

Aggregator metrics: `argus_fl_global_round`, `argus_fl_global_loss`, `argus_fl_global_accuracy`, `argus_fl_connected_clients`, `argus_fl_round_duration_seconds`, `argus_fl_client_{loss,accuracy,samples}{node_id}`, `argus_fl_model_version`, `argus_fl_checkpoints_total`, `argus_fl_rounds_total`, and, new, `argus_fl_rejected_updates_total{node_id}`. A node whose rejection count keeps rising is broken or hostile.

---

7. Runbook

Situation	Do this
A round is overdue	Check `GET /api/federated/status` and the node log (`journalctl -u argus-node`). Unreachable aggregator: it retries hourly by itself; fix the network or firewall. Too few labels: label more events
The new model behaves worse on camera	`GET /api/federated/head/versions` , then roll back to the last good label. Note the bad version's round directory in `data/fl_rounds/` for later
One node's updates are always rejected	Its data or software differs from the rest. Compare its `node_id` row in the screening reports; check its `samples` and loss
Run the FedAvg baseline for the study	Restart the aggregator with `-Aggregation fedavg` (or `ARGUS_AGGREGATION=fedavg`). Switch back afterwards
Start a round now	`POST /api/federated/round` with `{"server": "<aggregator>:8080"}`
Move the aggregator to a new machine	Copy `central_server/checkpoints/` ; the version counter and starting model resume from it

8. What still blocks calling this production

In order of risk.

Gap	Risk	Fix
The federated routes have no authentication, even with `AUTH_REQUIRED=true`	Anyone who can reach port 8000 can relabel data, start a round, or now roll the model back	Add `require_auth` to the routers in `src/api/trial_routes.py` and `dashboard_routes.py`, and probe one in `run_tunnel.sh`
Flower traffic is not encrypted	`start_server` runs without TLS certificates; weights cross the network in the clear and the server cannot tell a real node from an impostor	Flower's TLS (`certificates=` on the server, `root_certificates=` on clients), plus node authentication
No proof of origin for a model	A node trusts whatever the aggregator returns; only the gate stands between it and a tampered model	Sign checkpoints on the aggregator, verify on the node
The node does not record which global version it received	A node's `fl-r7` cannot be traced to the aggregator's `v12` except by timestamps	Return the global version in the evaluate config and store it in the gate decision
Robust screening is weak below three nodes	Direction and trimming are skipped, so a disagreeing update is not caught	Node B (parts by Oct 12); more nodes in deployment
Idle check never fed	"Only while idle" is not enforced	Call `notify_motion()` from the collector or worker
Aggregator history is in memory	Per-round reports are lost on restart (checkpoints are not)	Append each `RoundResult` to a JSONL file next to the checkpoints
No clip retention on nodes	The SD card fills; a card already corrupted once	Retention sweep (Mohammed, in progress)
One aggregator	If it is down, rounds wait (they retry hourly, so nothing is lost but time)	Acceptable for the project; note it

9. What has been verified

Claim	Evidence
Robust aggregation runs inside a real Flower server	`sim/fl.py serve` with three simulated clients over gRPC, two rounds, 2026-10-05: all updates screened and accepted, consensus applied
An attacker is left out; FedAvg is reproduced exactly when selected	`tests/test_robust_strategy.py` (11)
Failed rounds are retried, rejected rounds are not	`tests/test_fl_schedule.py` (2 new)
Versions archived, rollback restores weights and survives restart, unknown labels refused, routes work	`tests/test_model_versions.py` (6 node tests)
Aggregator version counter resumes and never goes backwards	`tests/test_model_versions.py` (2)
Loss tie-break	`tests/test_pipeline_changes.py` (2)
Whole suite	328 passed on 2026-10-05
Two real rounds Pi ↔ laptop	2026-09-20, before these changes

Not yet verified:

- The systemd unit on the Pi. Written, not installed.
- The Docker image with its new build context. Not built.
- A full 14-day cycle in the field. The schedule logic is tested; it has never been watched across a real interval.
- Any of the 2026-10-05 changes on the Pi or with two physical nodes.
- None of these changes are committed yet.